HACKATHON CONCEPT

ROOMRELAY

Book a room. Unlock access. Keep trust.

A room-sharing and access-management platform with NFC-enabled entry as an optional hardware layer.



PROJECT CONTEXT & BUILD BLUEPRINT

48-hour hackathon planning document | Version 1.0

Focused MVP: university classrooms, seminar rooms and meeting spaces

PRODUCT / WORKFLOWS / NFC / ARCHITECTURE

01 / OVERVIEW

RoomRelay at a glance

A focused product concept designed around the hackathon brief: unlock underused spaces while preserving owner control, safety and accountability.

THE ONE-LINE PITCH

RoomRelay helps universities publish available rooms, approve fair bookings and issue time-bound access permissions - with optional NFC tap-in at the door and a complete audit trail from request to exit.

Target setting

A university campus or institution with classrooms, seminar rooms, meeting rooms and student-club spaces.

Primary users

Students, club leads, faculty/staff, facilities administrators and authorised room managers.

Core value

Know what is free, who is allowed to book it, who approved it, whether access was granted and whether the room was returned properly.

Key differentiator

Booking is connected to real access events. NFC is the bridge between the reservation system and a physical room door.

What makes it more than a calendar?

A conventional room-booking calendar answers “When is the room free?” RoomRelay also answers “Who is eligible?”, “Who authorised this booking?”, “Can this person enter now?”, “Was the room accessed during the approved window?” and “What happened if the booking ran late or something went wrong?”

Suggested product boundary

Build a software-first room lifecycle that works end to end using a simulated or QR-based access check. Connect your existing NFC work as a real hardware proof-of-concept only if the reader/controller path is compatible and can be demonstrated reliably. The product must still be usable if the NFC hardware is disconnected.

Document map

Section	What it covers
Problem & users	Why room sharing fails; target personas and value proposition
Product & workflow	MVP boundaries, roles and the complete booking lifecycle
NFC & architecture	How access permission maps to a room booking; proposed software and hardware layers
Trust & fairness	Eligibility rules, allocation logic, privacy and security guardrails
Execution	Suggested stack, 48-hour plan, judge demo, testing and submission checklist

02 / PROBLEM AND USERS

The problem RoomRelay solves

The challenge is not simply that rooms go unused. It is that access to shared spaces is hard to coordinate safely and fairly.

The problem statement describes a mismatch: institutions have rooms and facilities that may sit idle, while students and teams struggle to find affordable, appropriate spaces. Basic calendars usually do not provide a complete control loop for eligibility, approvals, check-in, incidents, return confirmation and auditable decisions. RoomRelay narrows this broad resource-sharing problem to rooms.

Room owner / manager

Needs control over room availability, permitted use, approvals, cancellations, access windows and any incidents.

Student / club lead

Needs to discover suitable rooms, understand constraints, request a slot and know whether access is authorised.

Facilities administrator

Needs oversight across rooms, role permissions, policies, history and recurring misuse or scheduling conflicts.

Security / front desk

Needs a clear way to verify whether an entry attempt is authorised, without being asked to interpret a confusing booking screen.

Primary use case

Book a seminar room for a student club meeting

A club lead searches for a room for 12 people from 4:00-5:30 PM. RoomRelay checks capacity, availability and the club's eligibility. The room manager approves the request. A time-bound permission becomes active. At the door, the authorised person taps an assigned NFC credential or uses a demo fallback. Access is logged. After the meeting, the booking owner confirms the room is left in acceptable condition.

Why this is a strong hackathon scope

- Easy for judges to understand: everyone has experienced a room booking or access problem.
- A real end-to-end journey can be completed in the demo, from discovering a room to recording access and close-out.
- NFC adds a tangible technical element and differentiates the product from a static room calendar.
- The project directly maps to trust, role-based approval, fair allocation, conflict prevention and auditable history.

Problem statement to use in the project document

"Campus rooms are often managed across disconnected calendars, messages and manual access procedures. Students cannot reliably discover available spaces or know whether they are authorised to enter, while managers lack one consistent record of requests, approvals, room access and post-use issues. RoomRelay connects booking, approval and access into one accountable workflow."

03 / PRODUCT SCOPE

What to build - and what to avoid

The winning scope is a reliable room lifecycle, not a campus-wide facilities suite.

Build around three roles and one resource category first: students/club leads, room managers and administrators; rooms only. Add resource categories or multi-institution support only after the core workflow works.

Priority	Features	Reason
P0 - Required	Room listings with capacity, location, facilities, hours, availability and usage rules; search/filter; booking request; manager approve/reject; overlap prevention; booking status; access decision; check-in/out; audit history.	Delivers the required end-to-end journey.
P1 - Strong differentiator	NFC credential mapped to a user; time-window access checks; waitlist; fair-use limits; condition/incident report; overdue reminder; simple admin dashboard.	Proves the product is more than a calendar.
P2 - If time permits	QR fallback; room suggestions; email/in-app notifications; photo evidence; usage analytics; ICS calendar export.	Polish after reliability.
Later roadmap	Deep campus identity integrations; multiple institutions; payments/deposits; offline door controller; native mobile app; occupancy sensors; advanced AI matching.	High integration cost and not required to prove the concept.

Non-goals for the hackathon

- Do not attempt to support every building, every room type and every institution.
- Do not make an AI model the final authority on access, safety or penalties.
- Do not require judges to connect physical hardware before they can complete the core product journey.
- Do not claim production-grade door security from a demo NFC tag or a simulated unlock. Clearly label the hardware scope.
- Do not store personal details or booking permissions directly on an NFC tag. Use the tag as a credential reference; make the authorisation decision in trusted software.

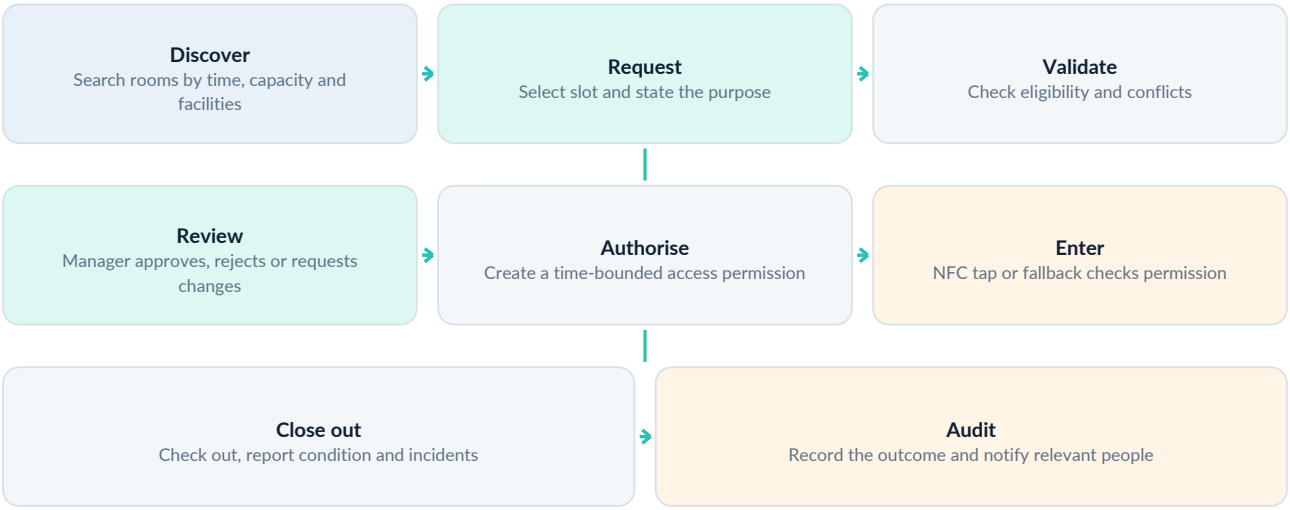
Definition of done

A judge can create or view a room, submit a booking, see a conflict or eligibility check, approve/reject the request as a manager, attempt access during the booking window, complete check-out/close-out, and inspect the resulting history without manual intervention by the team.

04 / LIFECYCLE

The complete RoomRelay workflow

Every state change should be visible to the people who need to act on it.



Recommended booking state model

State	Meaning	Allowed next step
Draft	User has not submitted request.	Submit or delete
Pending	Request submitted; manager decision required.	Approve, reject, request changes or waitlist
Approved	Room and time are allocated.	Cancel, check in during allowed window, or flag issue
Rejected	Manager declined; reason should be visible to requester.	User may edit/resubmit if policy permits
Waitlisted	No slot is available or fair-use policy defers the request.	Notify when a slot opens; promote transparently
Checked in	An authorised access event has started the booking.	Check out, extend request or report incident
Completed	Use period closed and return/condition confirmation recorded.	Review history; manager can append follow-up note
Cancelled / No-show / Incident	A non-standard outcome occurred.	Record reason and required resolution; keep history

Important edge cases

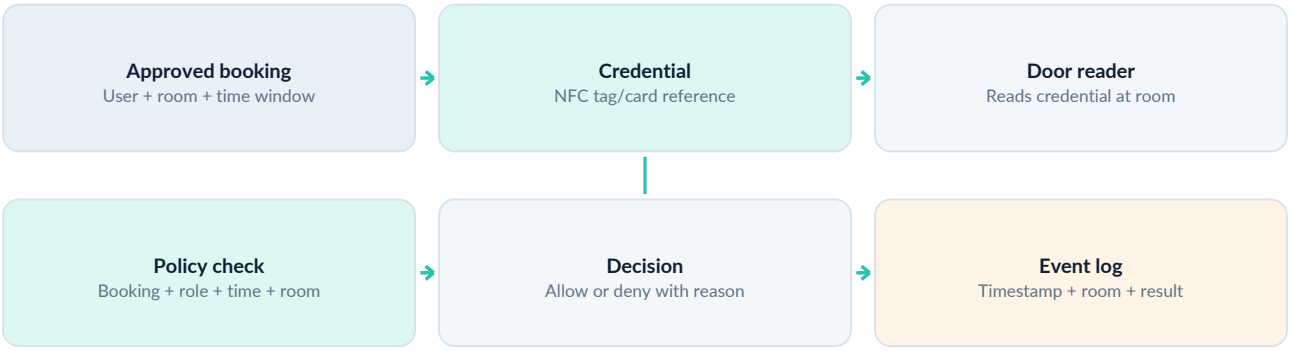
- A room is already booked: reject the overlapping slot or offer alternative times/rooms.
- A request arrives without required membership/training: show the missing requirement and do not auto-approve.
- The NFC tap happens outside the approved window: deny entry and show a clear reason in the log.
- The booking owner cancels: revoke future access permission and record the cancellation.
- The room is left damaged or over time: record the incident; route any penalty or sanction to a human reviewer.

05 / NFC ACCESS CONCEPT

NFC turns a reservation into an access decision

The physical access layer is a differentiator - but it should not be allowed to weaken the basic product.

Proposed model: a user is approved for a specific room and time window. The system associates the approved booking with an active user credential. At the door, an NFC reader captures a credential reference and sends the room/reader context to a controller or backend. The authorisation service checks the booking, user status, room mapping and current time before returning an allow/deny decision.



NFC design rules

Rule	Practical implication
NFC identifies a credential, not a person with certainty.	The app must bind the credential to a verified account and allow administrators to revoke it.
Do not trust a tag UID as the only production-grade secret.	UID-only tags can be copied or emulated in some environments. Treat a simple UID flow as a hackathon proof-of-concept, not hardened access control.
Authorise on the trusted side.	The controller sends a credential reference and reader/room ID; trusted application logic decides whether access is permitted. Do not encode names, email addresses or long-lived permissions on the tag.
Permissions must be time-bound and revocable.	A cancelled booking, disabled user or changed room policy must invalidate a future access attempt.
Fail safe and preserve egress.	A demo must not interfere with emergency exits or create a lock-in risk. Real doors need appropriate hardware, emergency egress and institutional approval.
Keep a fallback for the hackathon.	Provide a clearly labelled “Simulate NFC tap” or QR credential for judges who cannot access the hardware.

How to reuse your existing NFC project

Your earlier attendance/access project may offer reusable reader setup, tag-reading logic, microcontroller communication and event handling. Reuse these components only after checking hardware, library and protocol compatibility. Keep the RoomRelay booking and policy engine separate from reader firmware: hardware reports a credential scan; the application decides whether that scan is allowed for a room and time window.

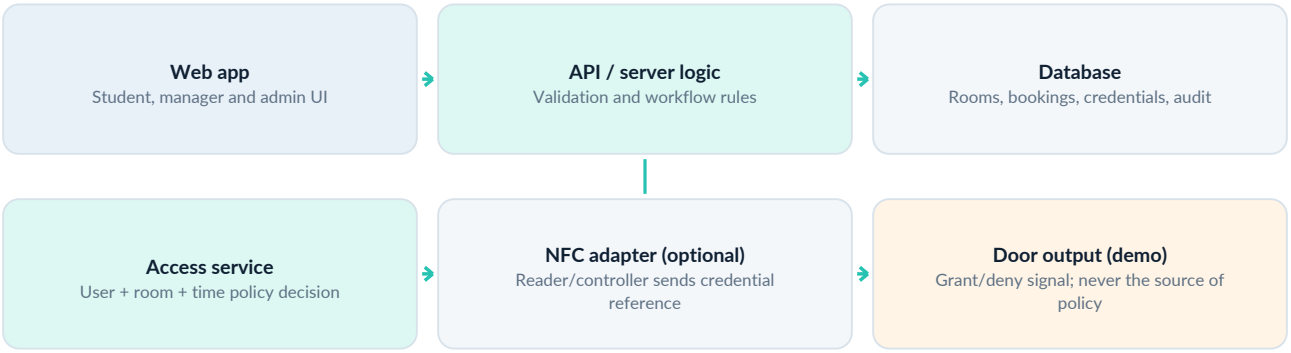
Recommended hackathon implementation order

- First: complete the software booking/access lifecycle using a simulated NFC event.
- Second: connect the compatible NFC reader/controller and map a tag to a demo user.
- Third: show a successful tap, a denied out-of-window tap and the associated event log.

06 / SYSTEM ARCHITECTURE

A small architecture with clear trust boundaries

Keep the app, decision logic and physical reader separate so the product remains testable without hardware.



Suggested stack for a 48-hour build

Layer	Suggested option	Why
Frontend	Next.js + React + TypeScript	Fast path to a responsive dashboard, room search and role-specific views.
UI styling	Tailwind CSS or an existing component library	Keeps layout consistent and saves build time.
Authentication	Supabase Auth or equivalent	Supports user sessions and role-aware app flows without building auth from scratch.
Database	PostgreSQL via Supabase	Relational data and constraints suit bookings, room ownership and audit events.
Server logic	Server routes / server actions / API endpoints	Booking overlap checks and access decisions remain server-side.
Photo evidence	Supabase Storage or equivalent (optional)	Stores condition/incident images with access policies.
NFC adapter	ESP32 / STM32 and compatible reader code from prior project	Hardware adapter publishes credential reference + room/reader identity.
Deployment	A publicly reachable hosted web app	Judges should not need a local development environment.

Architecture rule that prevents a common mistake

The browser must never be the authority for access. A client-side button can request an access check, but the server-side policy must validate the booking and produce the decision. A physical controller should accept only a well-defined response from the trusted decision layer, not simply unlock because the UI says “approved.”

Offline behaviour

Do not promise offline authorisation in the MVP unless you deliberately design, test and secure it. For the demo, show a clear service-unavailable state and preserve emergency egress. A future offline mode would need carefully bounded cached permissions, expiry, secure communication and reconciliation of queued events.

07 / DATA AND PERMISSIONS

Data model and role boundaries

The smallest robust design stores resources, bookings, access events and an append-only history of important changes.

Entity	Core fields (illustrative)	Purpose
users	id, display_name, email, role, status, created_at	Identity and account state; role should be verified server-side.
rooms	id, name, building, location, capacity, facilities, policy_id, manager_id, status	Discoverable room inventory and ownership.
availability_rules	room_id, start_time, end_time, recurrence, blackout_dates	When rooms can be booked.
bookings	id, room_id, requester_id, purpose, start_at, end_at, status, decision_reason	Reservation request and state.
eligibility_requirements	room_id or policy_id, requirement_type, description, required_flag	Membership, approval or training conditions.
credentials	id, user_id, credential_ref, credential_type, active, revoked_at	Server-side mapping from tag/token to account. Never store personal data on the tag.
access_events	id, credential_id, room_id, timestamp, result, reason, controller_id	Each NFC/simulated entry attempt.
condition_reports	booking_id, stage, notes, photo_refs, submitted_by, timestamp	Before/after condition and room handover.
incidents	booking_id, category, description, status, reviewer_id, resolution_note	Damage, delay, disputes and human-reviewed outcomes.
audit_events	actor_id, action, entity_type, entity_id, timestamp, summary, correlation_id	Trace important actions and decisions.

Role and permission matrix

Action	Student / club lead	Room manager	Admin
Browse rooms / policies	Yes	Yes	Yes
Create booking request	Own bookings	Optional	Optional
Approve / reject	No	Assigned rooms	All rooms / delegated
Edit room availability	No	Assigned rooms	All rooms
Bind / revoke NFC credential	View own status	No by default	Yes, audited
View access history	Own booking summary	Assigned rooms	Institution scope
Resolve incident / penalty	Submit / respond	Review / recommend	Final authorised review

Implementation notes

- Use database constraints or transaction-safe checks to prevent overlapping confirmed bookings for the same room.
- Do not accept a role sent by the browser as proof of permission; confirm it from trusted session/database state.
- Store timestamps consistently (for example, UTC in the database) and display them in the campus local timezone.
- Every approval, rejection, cancellation, permission change and incident resolution should create a history event.

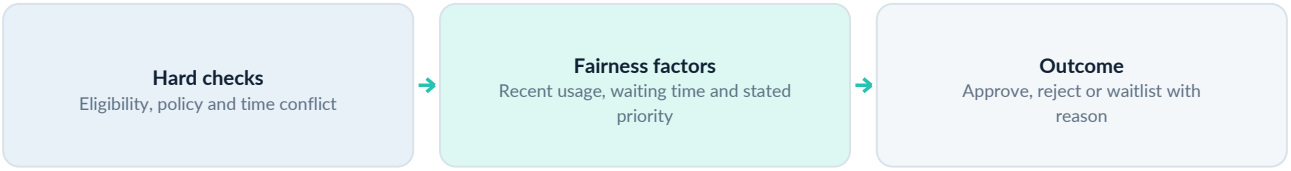
08 / FAIR ALLOCATION

Make room allocation transparent

Fairness is a product feature, not a hidden sorting formula.

Rooms can be scarce during club events, project reviews and exam periods. RoomRelay should make allocation rules visible in advance and explain the result after a decision. For the hackathon, use simple deterministic rules rather than opaque machine learning.

A practical allocation sequence



Rule	Behaviour	What user sees
Eligibility gate	A room with a mandatory requirement cannot be approved until the condition is satisfied or a manager overrides it with a recorded reason.	“Training verification is required for this room.”
Conflict prevention	A room cannot have two approved bookings whose intervals overlap. Define whether a setup/cleanup buffer is required.	“This slot conflicts with a confirmed booking. Try 5:00 PM or Room B.”
Transparent waiting list	When no slot is available, preserve request time and apply published priority rules.	“You are #2 on the waiting list; priority is based on eligibility, then wait time.”
Fair-use limit	Limit repeated prime-time reservations per user/group over a rolling period, where policy permits.	“Your group has reached its weekly prime-time quota.”
Explainable priority	Apply institution-defined priority categories only if the institution has actually approved them.	Show the exact policy factor that affected the result.
Human override	Managers may override a recommendation, but must select or enter a reason.	“Manager override: urgent faculty-led safety review.”

Do not over-engineer the fairness score

A transparent rule such as “eligibility first, then fewer recent prime-time bookings, then earliest request timestamp” is easier to test and defend than an invented AI fairness score. Decide policy before implementing the score. If the institution has no priority policy, present the factors as a demo policy, not as a universal definition of fairness.

AI: optional, assistive and reviewable

A small natural-language helper could map “need a room for 10 people with a projector after 3 PM” to filters. AI may summarise an incident report or suggest alternative rooms. It should not secretly approve entry, assess a person’s trustworthiness or impose penalties. Access and safety decisions should remain explainable and reviewable by an authorised person.

09 / PRODUCT EXPERIENCE

Screens the judges should be able to use

The UI should demonstrate one coherent story rather than many disconnected dashboards.

1. Room discovery

Search by date/time, capacity, building, projector/whiteboard and permitted use. Show availability and restrictions before the user starts a request.

2. Room detail

Show photos, capacity, facilities, manager, rules, eligible audiences, allowed hours and a clear request button.

3. Request form

Collect slot, duration, purpose/group, expected attendance and required acknowledgements. Validate obvious conflicts before submission.

4. My bookings

Upcoming, pending, approved, waitlisted, cancelled and completed states with next steps and clear reasons.

5. Manager queue

Pending requests with conflict/eligibility status, approve/reject controls, optional reason and a timeline of prior usage.

6. Access screen

Show current booking, credential status, time window, “simulate NFC tap”/QR fallback and the allow/deny result.

7. Check-out / report

Confirm room left as found, optionally attach condition evidence and submit an incident if needed.

8. Admin dashboard

Manage rooms, room managers, policies, users/credentials and audit events. Keep sensitive data limited by role.

Suggested visual direction

Use a clean campus-operations product aesthetic: deep navy navigation, teal action accents, white content surfaces, calm neutral backgrounds and status labels with both text and colour. Prefer practical room cards and timeline views over excessive gradients or decorative animation. Make booking states readable without relying on colour alone.

Key UI states that must exist

- Empty state: no rooms match; offer filter changes or alternative times.
- Pending state: make clear that the request is not yet permission to enter.
- Denied access: give a safe reason such as “No approved booking for this room at this time”; avoid leaking other users’ booking details.
- Conflict state: stop duplicate reservation and show available alternatives.
- Hardware unavailable: allow the judge to complete a simulated access check without pretending a real door unlocked.

10 / SECURITY AND SAFETY

Trust controls that matter

The product controls physical-space access. Even a prototype should show that access is more than a front-end toggle.

Area	Minimum control for the MVP	Important note
Authentication	Use a managed auth provider or seeded demo accounts with clear role separation.	Do not expose privileged admin actions to an ordinary user.
Authorisation	Server-side checks for role, room, booking status, time window and credential active/revoked status.	Never trust client-side “approved” state.
NFC credentials	Store only a credential reference/token mapping in the database; support revocation; do not write personal data to tags.	Treat UID-only demonstration as prototype-grade, not strong anti-cloning security.
Auditability	Record actor, action, entity, timestamp and reason for critical state changes. Avoid editing past events; append corrections.	Do not log secrets or unnecessary personal information.
Privacy	Collect the minimum information needed to process and audit a booking; restrict access history to appropriate roles.	Do not reveal another group’s purpose or membership unnecessarily.
Abuse resistance	Rate-limit repeated denied access checks; validate server payloads; protect admin routes; use secure secrets.	Avoid showing door-specific data to unauthorised users.
Physical safety	Keep emergency exits independent; do not use a prototype relay to create an unsafe lock. Have institution approval for any real door test.	A hackathon demo should not be installed on a live restricted door without authorisation.
Incident handling	Allow a report, status, reviewer and resolution note; require human review before penalties.	AI may assist summary, not make final sanctions.

NFC-specific threat model to mention in your README

- Cloned or shared tag: mitigate with backend verification, revocation, logs and stronger tag technology if production use requires it.
- Lost credential: allow an administrator to deactivate it immediately and issue a replacement.
- Cancelled booking after approval: invalidate access at the server, rather than trusting an old long-lived permission stored at the door.
- Replay or forged device request: authenticate communications between controller and server for real deployment; the hackathon adapter can use a documented test token.
- Network outage: be explicit that MVP access checks require the service; do not silently grant entry on failure. Keep emergency egress independent.

Claims to avoid

Do not call the product “unhackable”, “military-grade” or production-secure based on a tag UID. Say what is implemented, what is simulated, what is enforced server-side and what would be required before installation in a real campus.

11 / TECHNICAL INTEGRATION

Reuse your NFC work without coupling everything

Treat your previous NFC attendance/access project as an adapter to RoomRelay, not as the entire access-control system.

The saved project context indicates prior work with STM32/ESP32, Arduino IDE/STM32CubeIDE, and an ST25R3911AQB-W reader with an X-NUCLEO-NFC09A1 board. These components may be candidates for reuse, but exact code and compatibility should be verified before promising the hardware path.

Recommended separation

Component	Responsibility	Interface
RoomRelay web app	Room search, request, manager decisions, check-in/out, admin tools.	HTTP/API calls to server.
RoomRelay policy service	Checks booking state, room, time window, role, credential and revocation.	Input: credential reference + reader/room ID. Output: allow/deny + reason + event ID.
NFC reader firmware	Read the supported tag/card and produce a credential reference.	Serial/Wi-Fi/API message to a trusted controller/service, according to the hardware path.
Door controller (optional demo)	Receives a validated decision and signals a demo LED/relay output where safe.	Must not bypass policy. Use a bench demo, not an unauthorised live door.
Event logger	Persist each access attempt and decision.	Structured event from policy service; include correlation/event ID.

Integration plan

- **Phase A - no hardware:** create an AccessAttempt object in the UI; send demo credential ID + selected room; return a server-calculated decision and persist the event.
- **Phase B - reader integration:** have the reader/controller send the same event shape as Phase A. Keep the UI and database unchanged.
- **Phase C - physical output:** use a harmless bench indicator or mock lock output after a positive decision. Keep emergency egress and institution safety requirements out of scope for the demo.

Example access-check contract

```
POST /api/access/check
{
"credentialRef": "demo-tag-014",
"roomId": "room-seminar-02",
"readerId": "reader-demo-01",
"eventNonce": "unique-attempt-id"
}

Response (illustrative)
{
"decision": "DENY",
"reasonCode": "OUTSIDE_BOOKING_WINDOW",
"eventId": "access-event-2891"
}
```

This contract is illustrative. Adapt field names to your codebase; validate and authenticate all real device requests before deployment.

12 / 48-HOUR PLAN

Execution plan for a working product

Protect the core workflow first. Build optional hardware integration only after the software path is reliable.

Time window	Priority	Exit condition
Hours 0-3	Freeze use case, define roles/states, draw screens and data model, set up repo and deployment.	No ambiguity about booking states, rules or demo journey.
Hours 3-8	Build auth/demo roles, room seed data, room listing/detail and search/filter.	Judge can browse rooms and open a room detail.
Hours 8-14	Implement booking form, server-side eligibility and overlap checks, booking status.	User can submit request; duplicates are blocked.
Hours 14-19	Build manager queue, approve/reject with reason, cancellation and waitlist if time.	Request can be processed end to end.
Hours 19-24	Implement access policy endpoint, simulated NFC tap/QR fallback, time-window checks and access events.	Approved booking allows a valid attempt; invalid attempt is denied and logged.
Hours 24-29	Build check-out, condition/incident reporting, audit timeline and admin room setup.	Full lifecycle reaches completion with history.
Hours 29-34	Connect NFC reader adapter if feasible; test tag mapping and safe bench output.	Real tag event reaches the same policy check; fallback still works.
Hours 34-39	Polish UX, empty/error states, responsive layout and notifications.	No broken states in the judge path.
Hours 39-44	Run test cases, seed dependable demo data, fix critical bugs.	Fresh account/session can complete the demo reliably.
Hours 44-48	Record short backup demo, finalise README/project document, verify deployed links and submission.	Public/judge-accessible app, repo and complete documentation.

Work split if you have a teammate

Track A - product

Frontend pages, room discovery, booking experience, manager/admin views and visual polish.

Track B - platform

Schema, server-side booking rules, access decision endpoint, audit records, NFC adapter and deployment/integration support.

If you are solo

Cut NFC hardware integration before cutting the booking-to-return workflow. A complete simulated access flow, plus a small reader proof-of-concept if time remains, is stronger than a half-working hardware demo with no reliable web product.

13 / DEMO AND VALIDATION

The judge demo: tell one complete story

Target a 3-4 minute demo that works from a seeded fresh state. Keep the hardware path as a proof point, not a dependency.

Step	What the presenter does	What the judge should observe
1. Set the scene	Open the campus view and show rooms, facilities and availability.	RoomRelay solves a real room-discovery problem, not just a door unlock.
2. Request a room	As a student/club lead, request Seminar Room 02 for a defined time and purpose.	Eligibility is checked and the booking enters Pending.
3. Show manager control	Switch to Room Manager; inspect the request, then approve it. Optionally show an overlap rejection with a clear reason.	Approval is role-restricted; conflicts and decisions are explained.
4. Attempt access	Tap a mapped NFC tag or press the clearly labelled simulator/QR action.	Correct credential in approved window is allowed; an out-of-window or revoked credential is denied.
5. Close the booking	Check out and submit a “room left as found” condition report.	The resource lifecycle reaches Completed or an incident workflow.
6. Prove acc ountability	Open the audit timeline and show request, approval, access result and check-out event.	There is a record of what happened, who initiated it and why.

Must-pass test checklist

- A user can browse and filter rooms.
- A booking request persists and appears in the manager queue.
- A manager can approve/reject, and ordinary users cannot call manager-only actions.
- Two overlapping confirmed bookings for the same room cannot both succeed.
- Access is denied if there is no approved booking, the credential is revoked, or the current time is outside the allowed window.
- Each access attempt creates a timestamped event with a decision and reason.
- Cancellation revokes future access; historical events remain visible.
- A booking can be checked out/closed, and the result appears in history.
- The demo works without physical NFC hardware connected.

Demo resilience tip

Seed one room with a confirmed booking window that matches the demo, one pending request and one denied-attempt case. Keep demo accounts and credentials documented for judges. Avoid relying on email delivery, Wi-Fi hardware, or external integrations for the main story.

14 / EVALUATION AND DELIVERY

Map the solution to the challenge

Make it easy for a judge to see that RoomRelay meets the brief, while clearly disclosing what is simulated.

Judging criterion	How RoomRelay demonstrates it
Problem understanding - 15	Focused campus room use case with clear needs of requesters, room owners and administrators.
Core solution - 20	Discovery, eligibility, approvals, access checks, check-out and incident handling form one connected workflow.
End-to-end product - 20	A judge can complete the full lifecycle using the live app without a team member manually changing database state.
Technical depth - 15	Server-side conflict checks, role-based routes, time-bounded access policy, NFC adapter and event history.
Trust and safety - 10	Credential revocation, access decisions with reason, incident review, data minimisation and documented hardware limits.
Fairness and accessibility - 10	Transparent waitlist/usage-limit rules, meaningful status text, alternatives and readable responsive UI.
Originality and potential - 10	Connects booking operations to physical access events rather than treating a calendar as the whole product.

Submission checklist from the brief

- Public or judge-accessible deployed product link.
- Source-code repository with sufficient evaluation access.
- Project document describing use case, target users, evidence/reasoning, workflow, features, stack, architecture, AI/services, privacy/security/fairness, limitations and future work.
- Testing instructions and any demo credentials required by judges.
- Team/participant details and each member's contribution.
- Disclosure of templates, libraries, AI tools, assets and pre-existing code. Confirm the rules governing when the hackathon product may be developed.
- Keep the deployed app, repository and required links accessible throughout judging.

Short pitch for the submission

RoomRelay connects room discovery, fair booking and verified access

Universities often coordinate shared rooms through fragmented calendars, messages and manual access procedures. RoomRelay gives students a trusted way to discover and request rooms, gives managers control over eligibility and approvals, and connects approved bookings to an auditable access workflow. Its optional NFC layer lets an authorised user tap in during the allowed window, while every decision - from approval to denied entry and close-out - remains visible in a clear history. The hackathon MVP focuses on a reliable campus-room lifecycle, with transparent fairness rules and a software fallback when physical hardware is unavailable.

Recommended final scope statement

"RoomRelay is a university room-booking and access-management prototype. The MVP supports room discovery, time-slot requests, manager approval, conflict prevention, time-bound access checks, check-in/out and a complete audit history. NFC reader integration is an optional hardware adapter that reuses compatible components from an existing NFC project; any simulated path is labelled clearly. Real deployment would require institution approval, secure credential technology and a safety-reviewed door integration."